ASD24
Anti-patterns

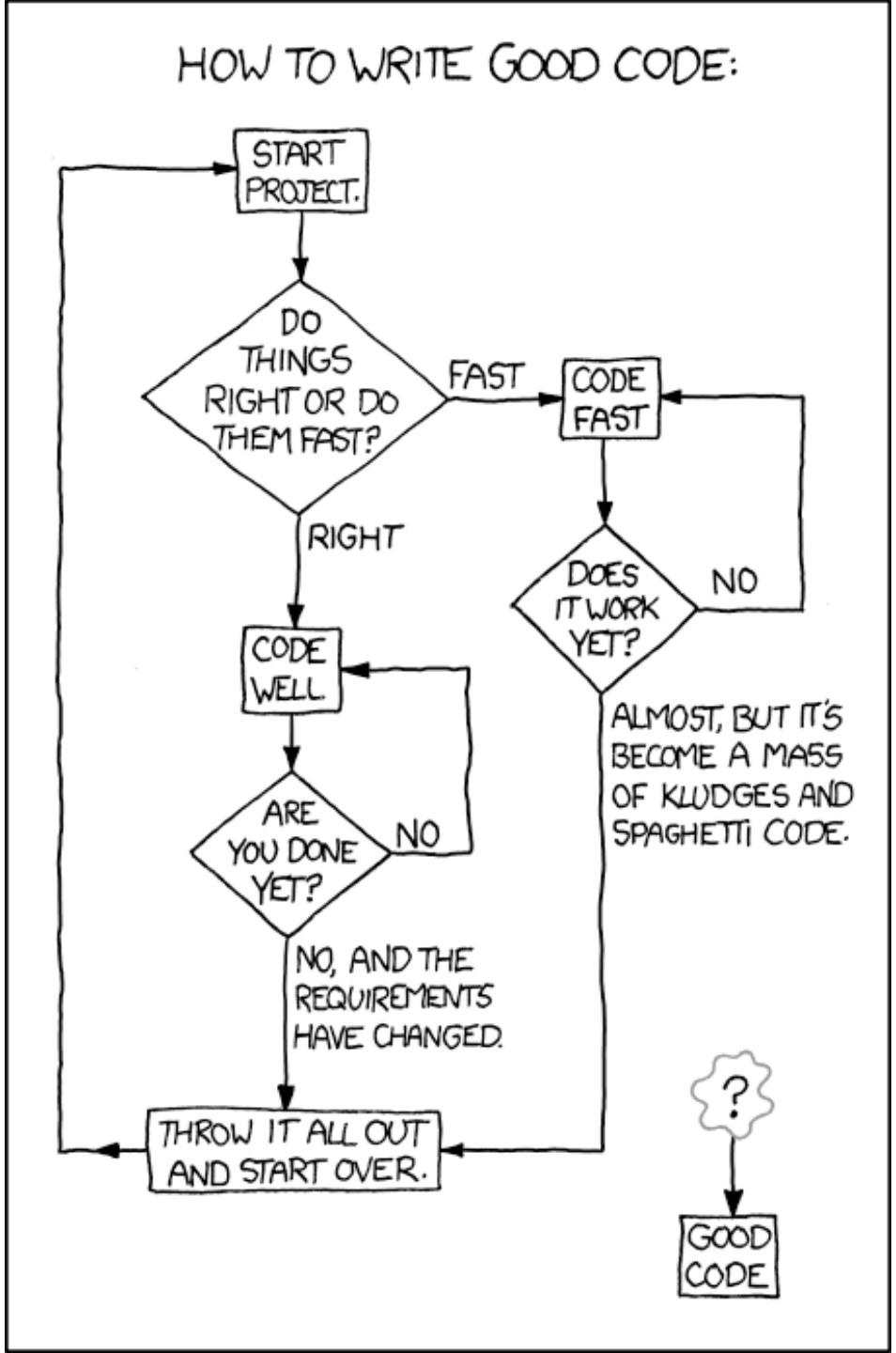
Johan Bontes

Antipattern

Andrew Koening (1995)

Common solution that is likely to be counterproductive

Pun on the GoF design patterns book



CSC3003S ASD24 Antipatterns

A. Generic software antipatterns

B. Object-oriented antipatterns

C. Programming antipatterns

D. Methodological antipatterns

Abstraction inversion

Problem: Users of construct need private functions

Wrong solution:

- Users re-implement the private functions in terms of public interface
- public interface uses (other) private functions

Result

- Lower-level features depend on higher-level ones thus the term 'abstraction inversion’.

Fix: Dependencies must be acyclic graph (hopefully a simple one)

Big ball of mud

Aka: spaghetti code/rat's nest

Edsgar Dijkstra: Goto considered harmful
Goto: jumps that place all code on a single level, no structure

Let to the development of structured programming and waterfall model

Problem: How to structure a program?

Wrong solution1: just get started and see where the flow takes us

Wrong solution2: use a system you do not understand

Result:

- Code that can only be understood as one complex unit
- It cannot be decomposed into independent items
- Hard to understand, untestable code

Fix: use design/architecture patterns + decomposition

Inner-platform effect

Problem:
Programmers want to add customisability to application

Wrong solution:
- Turn the application into a programming language

Result:
- Adds complex solution to perhaps a simple problem (addin will solve)
- Makes security very hard

Fix: Sandbox inner programming lang + just don’t: security nightmare

Examples:
- Plug-in based software (Firefox)
- MS Office

Stovepipe system

systems procured and developed to solve a specific problem, characterized by a limited focus and functionality, and containing data that cannot be easily shared with other systems.

Problem:

- Lack of architecture up front
- App was built to do one thing, but features got added

Wrong solution

- Reinvent the wheel for this one app only
- Do not use shared functionality from other system

Result:

- Lack of code re-use
- hard to fully test one-off systems

Fix: use standard components to extra func. (DB, etc)

Data races

Problem:

- Task parallel (multi-threaded) code is hard

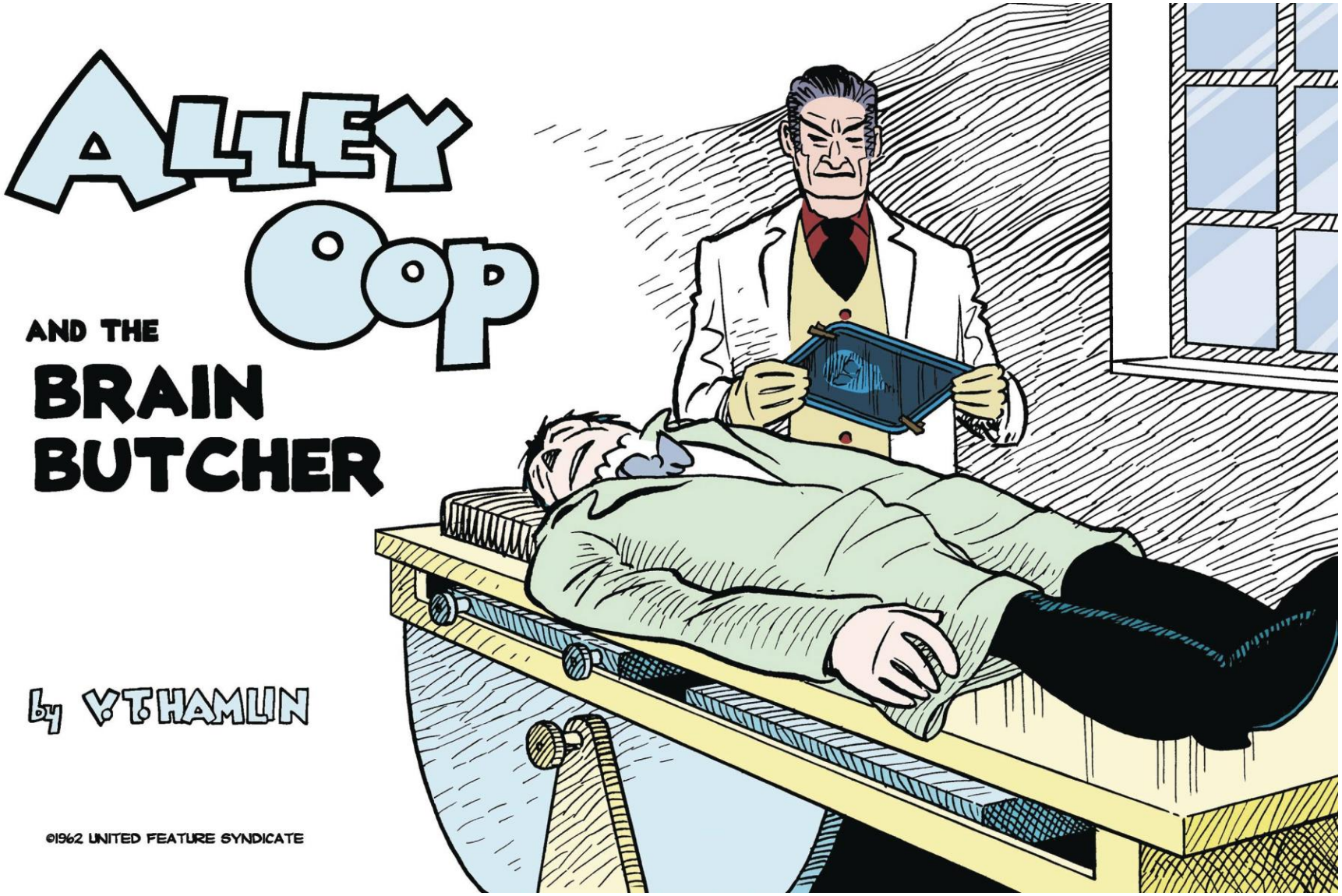
Wrong solution:

- Not realising that it is really hard
- Not using a known good solution

Result:

- Sequencing problems in Java (data corruption)
- Additionally: C++ Undefined behavior (bizarre side-effects)

Fix: Simplify threading model/ MPI / Use known good components (hard!)



CSC3003S ASD24 Antipatterns

A. Generic software antipatterns

B. Object-oriented antipatterns

C. Programming antipatterns

D. Methodological antipatterns

OOP: circular dependencies

Problem:

- Real world model has circular dependency, OOP models R.W.

Wrong solution:

- OOP circular dependency into classes

Result:

- SOLID violations, esp. tight coupling

Fix:

- Design patterns, e.g.: observer pattern

OOP: Circle – ellipse problem

Problem:

- Looks like an IS-A relationship, but invariants not the same

Wrong solution:

- Inheritance

Result:

- Liskov substitution violated

Fix: never violate Liskov

- Bad: Weaken restrictions
- Better: Use Mixin, not inheritance
- Programming languages: allow partial inheritance (e.g.: Lisp)

OOP: God object

Problem:

- Communication between objects = complex

Wrong solution:

- Everything in one class => easy communication

Result:

- Equivalent to spaghetti code in structured programming
- Hard to debug classes

Fix:

- Create proper hierarchy
- use patterns to keep communication simple

OOP: Sequential (temporal) coupling

Problem:

- Some steps need to happen in sequence (e.g.: init)

Wrong solution:

- Use of `init` or numbered methods

Result:

- Code misbehaves if earlier steps did not happen before later ones

Fix:

- Automatically call init/step 1 as needed in step 2.
- Use the template design pattern

OOP: Call super

Problem:

- Super class method has part of needed code
- Descendent method has other part of needed code
- OOP language makes calling super optional

Wrong solution:

- Specify ‘call super needed’ in the comments

Result:

- Code will break if parent method not called

Fix:

- Template design pattern
- LISP has ‘before’, ‘after’, ‘around’ requirements

OOP: unrestricted access (object orgy)

Problem:

- access to private members needed

Wrong solution:

- Make everything public

Result:

- Encapsulation broken
- No information hiding
- Huge ‘public’ interfaces (violates I in SOLID)

Fix:

- Properly design interface, break API if needed.



CSC3003S ASD24 Antipatterns

A. Generic software antipatterns

B. Object-oriented antipatterns

C. Programming antipatterns

D. Methodological antipatterns

Cargo cult programming

Problem:

- Use of paradigms / patterns without understanding

Wrong solution:

- More patterns

Result:

- Code does not make sense, hard to understand

Fix:

- CSC3003S
- Add experienced programmer, pair programming, code review

Coding by exception

Problem:

- Program flow is awkward

Wrong solution:

- Use an exception to break into subsection for non-exceptional code
- equivalent to using a GOTO

Result:

- Poor performance
- Hard to understand code
- Accidental hiding of errors (due to handled exceptions)

Fix:

- Read: goto considered harmful
- Use a nested function, or case statement, or redesign the logic, perhaps use GOTO

Repeated code

DRY

Shotgun surgery

Problem:

- Small changes needed in many places

Wrong solution:

- Perform lots of changes all over the place, quick and dirty

Result:

- Unit tests no longer current, docs out of date
- Non DRY code

Fix:

- Aspect oriented programming
- Refactor/rewrite code

Error hiding

Problem:

- Exceptions break your code

Wrong solution:

- Silently swallow exception

Result:

- Bugs/errors persist in code, but are undetectable

Fix:

- Investigate the source of the error, fix that
- Only handle exceptions that are well understood, or due to outside factors



CSC3003S ASD24 Antipatterns

A. Generic software antipatterns

B. Object-oriented antipatterns

C. Programming anti-patterns

D. Methodological antipatterns

Copy-paste programming

Problem:

- Answer was on stack overflow, or
- Worse, elsewhere in the code base
- Dunno why, but it seems to work

Wrong solution:

- Copy-paste solution as is, preferably in many places

Result:

- No DRY, undocumented code without unit tests
- Copy-paste code has hidden assumptions != your code

Fix:

- DRY + make sure you understand your code + assumptions in SO code

Copy-paste code2

Problem:

- Repetitive tasks occur, each one slightly different

Wrong solution:

- Copy paste code for task 1 many times, change for each variant

Result:

- Non DRY code, hard to read
- Poor performance due to overuse of the L1 code cache

Fix:

- DRY your code, isolate basic task in function
- C++: template is your friend
- Design patterns

Vibe-coding

See copy-paste programming x2

Premature optimisation

Problem:

- Code needs to be performant, and it may or may not be

Wrong solution:

- Design overly clever, fast *looking* solutions

Result:

- Unneeded complexity in the wrong places

Fix:

- Pick a good algorithm (with proper asymptotic runtime)
- Start with straightforward code
- Profile every iteration and optimise only where needed

Knuth: "We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical 3%"

Reinventing the (square) wheel

Problem:

- You have a problem, no-one else can solve it, you are the A-team

Wrong solution:

- You assume your problem is novel, it likely isn't
- You come up with a clever, novel solution

Result:

- You have wasted your time
- Possible on a suboptimal solution

Fix:

- Most problems already have solutions, use standard soln
- Check quality of given solutions